HAI913I · Évolution et restructuration des logiciels

TP 2 — Compréhension et architectures

Année 2026–2027 Enseignant Bachar Rima

OBJECTIF

Passer des **faits** extraits au TP 1 à une **structure de modules candidats** : calculer le **couplage** entre classes, en construire un **graphe pondéré**, regrouper les classes par **clustering hiérarchique ascendant**, en tirer un **dendrogramme**, le **couper** sous contraintes, puis **interpréter** le résultat.
Le CM 03 a montré cette chaîne et ses décisions ; ce TP la fait écrire. Il ne relit pas le code source, mais part plutôt de vos faits. Chaque étape se termine par un **point de contrôle** chiffré sur le projet de validation du TP 1.

PRÉREQUIS

- **CM 03** — dépendance choisie, couplage et dénominateur, cohésion, clustering ascendant, critère de liaison, dendrogramme, coupe sous contraintes, et la distinction *groupe* / *module* / *composant*.
- **TP 1** — votre modèle de faits et votre graphe d'appel. Ils sont l'entrée de ce TP, mais **ne pas les avoir terminés n'est pas bloquant** : voir § 4.
- **Java** — collections, tri, comparateurs, tests unitaires.

1. Ressources fournies

Toutes les ressources sont sur Moodle.

LES FAITS DU PROJET DE VALIDATION

- `resources/` — le **jeu de données canonique** : les faits validés du projet de validation du TP 1 (15 classes, 59 méthodes, 44 arcs d'appel), en **JSON** et en **CSV**. Son format est décrit dans `resources/README.txt`. C'est le chemin de repli décrit au § 4.
- `examples/` — un **micro jeu de données** de trois classes, dont toutes les valeurs attendues sont calculées à la main dans `examples/README.txt`, et deux petites démonstrations Java (formats de sortie, règle de départage).

POINT DE DÉPART ET ANNEXE

- `starter` (facultatif) — un squelette Maven **sans aucune dépendance d'exécution** : il lit le jeu de données canonique, affiche les compteurs du point de contrôle A0 et s'arrête là.
- **Annexe 1 « Regrouper des classes : implémentation, validation et restitution »** — l'architecture conseillée, le recalcul des similarités, les invariants à tester, les pannes fréquentes, et les formats de sortie exploitables.

2. Environnement technique

ÉLÉMENT	EXIGENCE POUR CE TP
JDK	**Java 17 au minimum**. Toutes les ressources sont validées sur **17.0.10** et **21.0.5**.
Construction	**Maven** recommandé (validé avec **3.9.6**). Gradle est accepté si le README donne des commandes équivalentes et testées.
Dépendances	**aucune n'est requise**. Le squelette fourni n'en a pas ; il lit les fichiers CSV avec la bibliothèque standard. Si vous préférez le JSON, ajoutez la bibliothèque de votre choix et indiquez sa version exacte dans le README.
Analyse du code source	**non requise**. Ce TP ne relit pas les `.java` : il part des faits du TP 1. Eclipse JDT et Spoon ne sont utiles qu'à un bonus.
Interface	une **ligne de commande**. Interface graphique en bonus.
Visualisation	**Graphviz** convertit un fichier DOT en image ; indiquez alors la version utilisée dans le README.

Environnement de développement. Il est **fortement recommandé** d'utiliser un environnement de développement intégré (IDE), notamment **Eclipse IDE** ou **IntelliJ IDEA** : navigation, débogueur, import d'un projet Maven, exécution des tests.

Il reste tout à fait possible de réaliser le TP avec un **éditeur de texte/code et les outils en ligne de commande** : Visual Studio Code, Sublime Text, Vim, Emacs ou tout environnement équivalent, combinés à `mvn`, `javac` et `java`. Aucun élément du sujet ne dépend d'un IDE, et les deux approches sont évaluées de la même façon.

> **NOTE**
> Quel que soit votre choix, vous devez pouvoir expliquer *ce que votre outil fait à votre place* : quelle commande est lancée, depuis quel **répertoire de travail**, et où sont lus les fichiers de données passés en argument.

3. Modalités de travail

Travail en binôme. Le TP 2 se réalise et se rend **en binôme**. Le travail en monôme reste possible **à titre exceptionnel**, pour qui n'a pas pu intégrer un binôme ; prévenez alors l'enseignant. Dans un binôme, **chacun des deux membres** doit pouvoir expliquer l'ensemble du rendu, et pas seulement « sa » partie. Discuter d'un problème avec d'autres binômes est normal ; rendre du code que l'on ne saurait pas expliquer ne l'est pas.

3.1. Usage des outils d'intelligence artificielle générative

L'usage d'outils d'IA générative est **autorisé**, à condition qu'il reste un *appui* et qu'il soit *déclaré*.

USAGES LÉGITIMES	CE QUI N'EST PAS ACCEPTABLE
Faire expliciter une notion du cours (lien moyen, dendrogramme, coupe).	Faire produire tout ou partie de l'algorithme sans le comprendre.
Demander des exemples supplémentaires.	Rendre du code que vous ne sauriez pas justifier.
Comprendre un message d'erreur, chercher pourquoi deux exécutions donnent deux arbres différents.	Présenter comme vôtres des conventions ou une conception que vous n'avez pas examinées.
Discuter des alternatives de conception, organiser des idées, améliorer la rédaction d'une explication, localiser une documentation.	Produire un compte-rendu dont le contenu ne correspond pas à votre code ou à votre vidéo.

Vous restez **entièrement responsables** de l'architecture, de l'implémentation, de la correction des résultats, de leur vérification et des explications que vous rendez. Vous devez pouvoir **expliquer et défendre** chaque élément significatif de votre travail.

CE SUR QUOI PORTE RÉELLEMENT L'ÉVALUATION

Il n'existe aucun moyen fiable de détecter automatiquement du texte ou du code engendré par une IA, et ce TP ne cherche pas à le faire. L'évaluation porte sur la **transparence**, la **compréhension démontrée**, la **cohérence entre le compte-rendu, le code et la vidéo**, votre capacité à **justifier vos choix** et la **reproductibilité** de votre travail. Un dossier cohérent et assumé, avec une déclaration honnête, n'est jamais pénalisé par le fait d'avoir utilisé un assistant.

Le compte-rendu comporte une section dédiée : voir § 6, point **A.7**.

4. D'où viennent les faits : deux chemins

Le TP 2 dépend de quelques résultats du TP 1. Un TP 1 encore incomplet ne doit donc pas empêcher de faire le TP 2. Deux chemins sont ouverts, et ils sont **équivalents pour l'évaluation**.

CHEMIN	CE QUE VOUS BRANCHEZ	CE QUE CELA DEMANDE
1 — le vôtre *recommandé*	votre propre modèle de faits du TP 1 : classes, méthodes, graphe d'appel	écrire un **adaptateur** qui expose vos faits derrière la petite interface d'entrée du TP 2 ; rien d'autre du TP 2 ne doit connaître vos classes du TP 1
2 — le repli	le jeu de données canonique fourni sur Moodle, qui contient les mêmes faits, validés	écrire un **chargeur** (ou réutiliser celui du squelette) ; le reste du TP 2 est identique

CE QUE LE JEU DE DONNÉES CANONIQUE NE CONTIENT PAS

Aucun couplage, aucune matrice, aucun regroupement, aucun dendrogramme, aucun module. Il ne contient pas non plus de code d'extraction : ni analyseur, ni AST, ni parcours de sources.

INDICE

Si vous utilisez le chemin 2, **déclarez-le** dans le compte-rendu et le README. Le barème ne distingue pas les deux chemins.

5. Conventions imposées

Un regroupement n'a de sens qu'avec les définitions qui l'ont produit. Les conventions ci-dessous sont **imposées** et rendent les points de contrôle vérifiables et les résultats comparables d'un binôme à l'autre. Vous pouvez calculer d'autres variantes *en plus*, à condition de les nommer et de les définir.

5.1. La relation de dépendance comptée

UNE SEULE RELATION : L'APPEL DE MÉTHODE

Deux classes sont en relation lorsqu'une méthode de l'une **appelle** une méthode de l'autre. C'est la définition du CM 03. Elle est **volontairement réductrice** : l'héritage, l'implémentation d'une interface, le type d'un attribut, le type d'un paramètre ou d'un retour, l'instanciation (`new`) et la levée d'une exception **ne comptent pas**.

Vous en observerez les conséquences sur le projet de validation, et vous **devez les discuter** (étape B4). Par exemple, deux classes y sont invisibles pour cette définition, et une troisième n'est reliée qu'à travers l'interface qu'elle implémente. Un bonus permet d'enrichir la relation *et de comparer* (voir la **partie C**).

5.2. Le couplage entre deux classes

DÉFINITION RETENUE DU CM 03

Pour deux classes **distinctes** A et B :

$$C(A,B) \;=\; \frac{r(A,B)}{|E_{\text{inter}}|}$$

- $r(A,B)$: nombre d'**arcs** du graphe d'appel reliant une méthode de A à une méthode de B, **dans un sens ou dans l'autre** ;
- E_{inter} : l'ensemble des arcs du graphe d'appel dont les deux extrémités appartiennent à **deux classes distinctes** ; $|E_{\text{inter}}|$ est son nombre d'éléments ;
- les arcs **internes** à une classe ($A = B$) sont écartés : ils relèvent de la cohésion, pas du couplage ;
- $C(A,A) = 0$ par convention, et $C(A,B) = C(B,A)$; au niveau des classes, **le sens de l'appel est perdu**.

ARCS, ET NON SITES D'APPEL

Un **arc** est un couple (méthode appelante, méthode appelée), compté **une seule fois**, même si l'appel apparaît à plusieurs endroits du corps. Un **site d'appel** est chaque occurrence écrite dans le code. Le TP 2 compte des **arcs**. Sur le projet de validation, la différence est mesurable : 37 arcs inter-classes contre 38 sites d'appel inter-classes, soit $C(\texttt{Item},\texttt{LoanService}) = 4/37 = 0{,}1081$ au lieu de $4/38 = 0{,}1053$. Compter les sites est une convention *alternative* défendable mais elle ne doit pas remplacer la convention retenue.

Deux conséquences utiles pour vous auto-vérifier : la somme de $C(A,B)$ sur **tous** les couples de classes distinctes vaut exactement 1, et aucune valeur ne dépasse 1. Les valeurs sont donc **petites** ; sur le projet de validation, le couple le plus couplé est à $0{,}1081$. Ainsi, un seuil « couplage $> 0{,}5$ » n'aurait aucun sens ici.

5.3. Le critère de liaison : lien moyen

Après une fusion, il faut une similarité entre le nouveau groupe et chacun des autres. Le TP 2 impose le **lien moyen** (*average linkage*) : pour deux groupes de classes G_1 et G_2 disjoints,

$$S(G_1,G_2) \;=\; \frac{1}{|G_1| \cdot |G_2|} \sum_{a \in G_1} \sum_{b \in G_2} C(a,b)$$

- la somme porte sur **tous** les couples inter-groupes, **y compris ceux dont le couplage est nul** qui font baisser la moyenne quand un groupe grossit ;
- pour deux groupes réduits à une classe, S redonne C ;
- le **lien simple** (le meilleur couple) et le **lien complet** (le pire couple) sont d'autres choix légitimes ; comparer le lien moyen avec le lien simple est un bonus (**C1**).

5.4. Les égalités doivent être départagées

Plusieurs couples de groupes atteignent régulièrement la **même** similarité maximale. Sur le projet de validation, cela arrive à **la moitié des étapes**, avec jusqu'à trois couples à égalité. Sans règle, deux exécutions du même programme peuvent produire deux arbres différents.

RÈGLE DE DÉPARTAGE IMPOSÉE

1. **Chaque groupe actif porte un identifiant entier stable.** Les M classes initiales sont numérotées de 0 à $M-1$ par **ordre alphabétique de nom qualifié** (`library.app.LibraryApp` avant `library.model.Book`…). Chaque groupe créé par une fusion reçoit l'identifiant suivant : M, puis $M+1$, etc.
2. **Un couple de groupes s'écrit toujours dans le même sens**, du plus petit identifiant vers le plus grand : $\big(\min(i,j),\ \max(i,j)\big)$.
3. **À égalité de similarité maximale, on compare les couples lexicographiquement** : d'abord le premier identifiant ; s'ils sont égaux, le second.
4. **On retient le plus petit couple.**

LA RÈGLE APPLIQUÉE

Trois couples atteignent la même similarité maximale $0{,}25$:

$$(0, 2) \qquad (1, 3) \qquad (2, 3)$$

On compare les *premiers* identifiants : $0 < 1 < 2$, donc le couple retenu est $(\mathbf{0}, \mathbf{2})$.
Si les premiers identifiants sont égaux, on compare les seconds. Entre $(0, 2)$ et $(0, 3)$: même premier identifiant 0, et $2 < 3$, donc $(\mathbf{0}, \mathbf{2})$ l'emporte.

Cette règle n'est pas un meilleur choix de fusion, c'est une **convention déterministe**. Elle garantit trois choses : deux exécutions de votre programme donnent le même arbre, votre binôme obtient le même arbre que vous, et cet arbre est celui de l'exemple filé du CM 03. La démonstration `TieBreakDemo`, dans `examples/`, l'applique à une matrice contenant trois couples à égalité.

COMPARER DES FLOTTANTS

Deux similarités mathématiquement égales peuvent différer du dernier bit selon l'ordre des additions. Comparez avec une **tolérance** (par exemple 10^{-9}), jamais avec `==`. Sinon une égalité passe inaperçue, la règle de départage ne s'applique pas, et le résultat cesse d'être reproductible.

5.5. Les modules

Le dendrogramme ne désigne aucun module ; il faut le couper. Les trois contraintes ci-dessous sont celles du CM 03.

CONTRAINTE	ÉNONCÉ
Nombre de modules	l'application contient **au plus** $M/2$ **modules**, où M est le nombre de classes ; $M/2$ en **division entière** ($M = 15 \Rightarrow$ au plus 7).
Forme d'un module	chaque module retenu est une **branche** du dendrogramme, c'est-à-dire l'ensemble des feuilles d'un de ses sous-arbres. Un regroupement arbitraire de classes n'est pas un module.
Couplage interne moyen	le **couplage interne moyen** du module est **strictement supérieur** au seuil `CP`. Un module d'une seule classe n'a aucun couple : la contrainte ne s'applique pas à lui, et la feuille est retenue telle quelle.

Le **couplage interne moyen** d'un groupe G d'au moins deux classes est la moyenne du couplage sur tous ses couples de classes :

$$\overline{C}(G) \;=\; \frac{1}{\binom{|G|}{2}} \sum_{\{A,B\}\subseteq G,\; A\neq B} C(A, B)$$

où $\binom{|G|}{2}$ est le nombre de couples de classes du groupe. La condition de coupe s'écrit alors $\overline{C}(G) > \texttt{CP}$: **strictement** supérieur, jamais « supérieur ou égal ».

LE RÔLE DU SEUIL CP

La coupe est un **parcours de l'arbre depuis la racine**. À chaque nœud, on a une seule question à se poser : le couplage interne moyen $\overline{C}(G)$ dépasse-t-il `CP` ?

- **oui** : le nœud devient un module, et on **arrête** de descendre dans cette branche ;
- **non** : on descend dans **les deux** fils ;
- une **feuille** est retenue telle quelle, comme module d'une seule classe.

Une fois la coupe terminée, on vérifie la contrainte globale $M/2$.

D'où le sens de variation :

- `CP` **bas** $\Rightarrow$ exigence faible $\Rightarrow$ les nœuds sont acceptés *haut* dans l'arbre $\Rightarrow$ modules **peu nombreux et gros** ;
- `CP` **élevé** $\Rightarrow$ exigence forte $\Rightarrow$ le parcours descend *plus bas* $\Rightarrow$ modules **nombreux et petits**, jusqu'à rendre la contrainte $M/2$ impossible à satisfaire.

`CP` **calibre donc la granularité** du découpage. Il ne prescrit **pas** le nombre de modules mais le contraint indirectement. Et il n'a **pas de valeur par défaut**, parce qu'il s'applique à des couplages dont l'échelle dépend de la mesure et du système analysé : sur le projet de validation, le couple le plus couplé est à 0,1081.

5.6. Vocabulaire : ce que vous produisez

UN GROUPE DE CLASSES N'EST PAS UN COMPOSANT

Ces quatre mots ne sont pas interchangeables. Pour ce TP :

- **groupe** (*cluster*) : **tout** nœud produit pendant la construction du dendrogramme, de la feuille à la racine. Il y en a $2M - 1$, et la plupart ne seront jamais retenus.
- **module** : un groupe **retenu par la règle de coupe** (§ 5.5). C'est ce que votre programme doit produire, et c'est le mot à employer par défaut dans votre compte-rendu.
- **composant candidat** : un module auquel **vous** avez attribué, par interprétation, un rôle architectural — un nom, des interfaces fournies et requises, une justification. Rien dans ce TP ne l'exige : c'est une décision humaine, pas une sortie de calcul.
- **composant déployable**, **microservice** : hors programme.

PARTIE A Des faits au graphe de couplage

OBLIGATOIRE · COMPTE POUR LA NOTE SUR 20

Cette partie transforme les faits du TP 1 en une **relation pondérée entre classes**. Elle se construit étape par étape ; **n'attaquez pas une étape tant que le point de contrôle de la précédente n'est pas franchi**. Un regroupement absurde vient presque toujours d'un couplage faux.

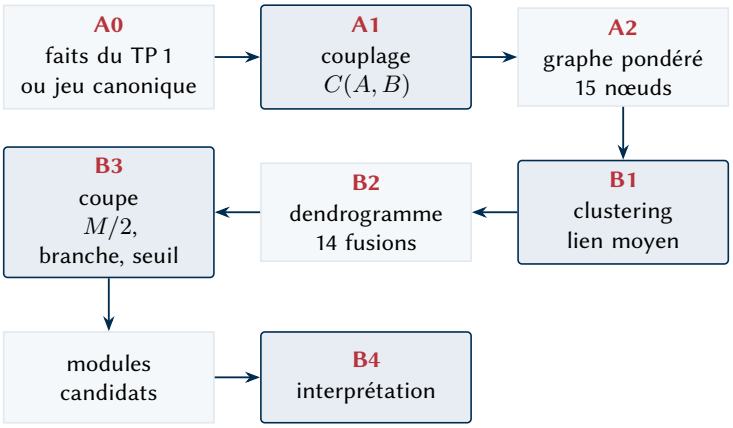


Figure 1 – La chaîne des étapes du TP 2.

A0 Environnement et chargement des faits

TRAVAIL DEMANDÉ

1. Vérifiez `java -version` (17 ou plus) et `mvn -v`.
2. Créez votre projet : partez du `starter`, ou d'un projet Maven vide.
3. Choisissez votre chemin d'entrée (§ 4) et **isolez-le derrière une seule interface** ; le reste de votre programme ne doit manipuler que des classes, des méthodes et des arcs.
4. Chargez les faits et affichez les trois compteurs : nombre de classes, de méthodes, d'arcs du graphe d'appel.

5. Faites de même sur le micro jeu de données `examples/facts-mini` et écrivez vos premiers tests ; ses valeurs sont calculées à la main dans `examples/README.txt`.

```
$ cd starter
$ mvn clean package
$ java -jar target/hai913i-tp2.jar ../resources
Dossier des faits   : ...\resources
Jeu de données      : facts-validation
Classes             : 15
Méthodes            : 59
Arcs du graphe      : 44
Seuil CP retenu     : 0.0250
```

INDICE

Même si deux classes du projet n'apparaissent dans **aucun** appel, elles doivent pourtant compter dans M et figurer dans le résultat final. Ne déduisez donc **pas** la liste des classes du seul fichier des appels.

POINT DE CONTRÔLE A0

Votre programme annonce **15 classes**, **59 méthodes** et **44 arcs** sur le jeu de données du projet de validation, et **3 classes**, **5 méthodes**, **4 arcs** sur le micro jeu de données. Si vous utilisez le `starter`, `mvn clean test` passe.

A1 Du graphe d'appel au couplage entre classes

TRAVAIL DEMANDÉ

1. Écartez les arcs **internes** à une classe et comptez $|E_{\text{inter}}|$.
2. Agrégez les arcs restants par **couple de classes distinctes**, sans tenir compte du sens pour obtenir $r(A, B)$.
3. Calculez $C(A, B) = r(A, B)/|E_{\text{inter}}|$ pour tous les couples, et rangez ces valeurs dans une structure que la suite pourra interroger efficacement (une matrice, ou une table indexée par couple trié).
4. Vérifiez la symétrie, la diagonale nulle, et que la somme des couplages vaut 1.
5. Affichez les couples par couplage décroissant.

INDICE

Le piège classique : compter **deux fois** l'arc $A \to B$ et l'arc $B \to A$ comme s'il s'agissait de deux couples différents.

POINT DE CONTRÔLE A1

Sur le projet de validation : **7 arcs internes** à une classe sont écartés, donc $|E_{\text{inter}}| = \mathbf{37}$. Exactement **19 couples** de classes sur les **105** possibles ont un couplage non nul. La matrice est symétrique et de diagonale nulle. Le couple le plus couplé est `Item` – `LoanService` avec $r = 4$, soit $C = 4/37 = \mathbf{0{,}1081}$. La somme des 19 couplages vaut **1,0000**. Les classes `Dvd` et `LoanException` n'apparaissent dans **aucun** couple.

A2 Le graphe de couplage pondéré

TRAVAIL DEMANDÉ

1. Construisez le **graphe pondéré** : un nœud par classe, une arête non orientée par couple de couplage non nul, le poids étant $C(A, B)$.

2. Exportez-le dans un format **relisible** : un format texte que votre propre programme sait recharger (CSV, JSON…) et un export DOT (`examples/` en montre un).
3. Faites afficher les cinq couples les plus couplés, et la liste des classes **isolées** (aucune arête).

INDICE

Le graphe est **très creux** : 19 arêtes pour 105 couples possibles. Ne stockez pas une matrice dense si cela vous gêne ; une table des seuls couples non nuls suffit, à condition que la lecture d'un couple absent renvoie **0** et non une erreur.

POINT DE CONTRÔLE A2

Le graphe a **15 nœuds** et **19 arêtes**. Les cinq arêtes les plus lourdes sont `Item`–`LoanService` et `LoanService`–`Loan` $(4/37)$, puis `LibraryApp`–`Item`, `LibraryApp`–`Catalog` et `LibraryApp`–`LoanService` $(3/37)$. Deux nœuds sont isolés : `Dvd` et `LoanException`. Le fichier exporté se recharge et redonne les mêmes valeurs.

PARTIE B Regrouper, couper, interpréter

OBLIGATOIRE · COMPTE POUR LA NOTE SUR 20

Cette partie est le cœur du TP. Elle construit l'arbre de regroupement, en tire des modules, et **les interprète**.

B1 Le clustering hiérarchique ascendant

TRAVAIL DEMANDÉ

1. Implémentez le regroupement ascendant : au départ, **chaque classe forme un groupe** ; à chaque itération, fusionnez les **deux groupes les plus similaires** ; recommencez jusqu'à n'avoir qu'un seul groupe.
2. Après chaque fusion, **recalculez** la similarité du nouveau groupe avec tous les autres, selon le **lien moyen** (§ 5.3).
3. Appliquez la **règle de départage** des égalités (§ 5.4).
4. Conservez, pour chaque fusion : les deux groupes réunis et la **similarité au moment de la fusion**.
5. Exécutez deux fois votre programme et comparez les sorties octet par octet.

INDICE

Deux erreurs fréquentes (détaillées en annexe 1), toutes deux invisibles sur de petits jeux de données :

- oublier les couples de couplage **nul** dans la moyenne alors que le dénominateur est $|G_1| \cdot |G_2|$, pas le nombre de couples non nuls.
- recalculer la similarité à partir des **anciennes similarités de groupes** au lieu des couplages entre classes.

POINT DE CONTRÔLE B1

Sur le projet de validation, votre programme effectue exactement $M - 1 =$ **14 fusions**, et les similarités de fusion **ne remontent jamais** d'une étape à la suivante. Elles peuvent en revanche être égales, et les deux dernières valent 0. Les trois premières sont :

ÉTAPE	GROUPE A	GROUPE B	SIMILARITÉ
1	{`Item`}	{`LoanService`}	0,1081
2	{`LibraryApp`}	{`Catalog`}	0,0811
3	{`Item`, `LoanService`}	{`LibraryApp`, `Catalog`}	0,0608

L'étape 3 est vérifiable à la main : les quatre couples inter-groupes valent 3, 3, 2 et 1 arcs, d'où $(3 + 3 + 2 +$

$1)/(4 \times 37) = 9/148 = 0{,}0608$. À chaque étape, les groupes actifs forment une **partition** des 15 classes. Deux exécutions donnent la **même** suite de fusions.

B2 Le dendrogramme

TRAVAIL DEMANDÉ

1. Représentez l'arbre complet en mémoire : une feuille par classe, un nœud interne par fusion, portant ses deux fils et la similarité de la fusion.
2. Exportez-le sous une forme lisible **sans besoin d'exécuter votre code** : arbre indenté, liste de fusions, JSON, etc. Le format doit laisser voir **l'ordre des fusions, les membres de chaque groupe et la similarité de chaque fusion**.
3. Produisez une image (Graphviz) du résultat obtenu.

INDICE

Le dendrogramme du projet de validation est très « plat » : la plus forte fusion est à $0{,}1081$ et les dernières sont à $0{,}0025$ et à 0. C'est normal pour une mesure normalisée sur 37 relations.

POINT DE CONTRÔLE B2

L'arbre a **15 feuilles** et **14 nœuds internes**. Les membres de tout nœud interne sont exactement l'union **disjointe** des membres de ses deux fils. La racine contient les 15 classes, chacune **une seule fois**. Les **deux dernières** fusions se font à une similarité de **0**.

B3 Identifier les modules

TRAVAIL DEMANDÉ

1. Implémentez la coupe : parcourez l'arbre depuis la racine ; si le **couplage interne moyen** $\overline{C}(G)$ du nœud dépasse `CP`, le nœud devient un module et on arrête de descendre dans cette branche ; sinon, descendez dans ses **deux** fils. Une feuille est toujours un module d'une seule classe.
2. Vérifiez ensuite la contrainte $M/2$, et **signalez clairement** le cas où elle est violée.
3. Rendez `CP` paramétrable sur la ligne de commande et **essayez plusieurs valeurs**.
4. Affichez, pour chaque module : ses classes, leur nombre, et son couplage interne moyen.

LE COUPLAGE INTERNE MOYEN N'EST PAS LA SIMILARITÉ DE LA FUSION

La contrainte porte sur $\overline{C}(G)$, la moyenne du couplage sur **tous les couples de classes du module**, soit $\binom{n}{2}$ couples pour un module de n classes. C'est une quantité **interne** au module. La similarité de la fusion qui a créé ce nœud, en revanche, est une quantité **entre** ses deux fils. Les deux quantités sont différentes et les confondre est l'erreur la plus fréquente de cette étape.

POINT DE CONTRÔLE B3

Sur le projet de validation, $M = 15$ donc au plus **7 modules**.

SEUIL CP	MODULES	CONTRAINTE $M/2$
0,015	**4**	respectée, mais on obtient un module de onze classes (trop grossier)
0,025	**7**	respectée, **tout juste** : $7 = M/2$
0,045	**10**	**violée** : la coupe doit être rejetée

Pour chaque valeur, les modules forment une **partition** des 15 classes (aucune classe oubliée, aucune en double), chacun correspond à une **branche** du dendrogramme, et tout module de deux classes ou plus a un couplage interne moyen **strictement supérieur** à `CP`.

B4 Interpréter et critiquer le résultat

TRAVAIL DEMANDÉ

Dans le compte-rendu, traitez explicitement les cinq points suivants, en vous appuyant sur **vos** sorties.

1. **Modules et paquetages.** Comparez la partition obtenue pour votre `CP` retenu avec les paquetages réels du projet (`library.app`, `library.model`, `library.service`, `library.util`). Où coïncident-ils ? Où divergent-ils ? Que suggère chaque écart : une érosion de l'architecture, une faiblesse de la mesure, ou simplement deux préoccupations différentes ?
2. **Les classes isolées.** Pourquoi `Dvd` et `LoanException` restent-elles isolées et deviennent-elles des modules d'une seule classe avec cette relation de dépendance ? Remontez à la définition du § 5.1 plutôt qu'à l'algorithme.
3. **Un groupe qui surprend.** Choisissez un module dont la composition vous paraît discutable et expliquez pourquoi l'algorithme l'a formé.
4. **L'effet de `CP`.** Donnez les résultats pour au moins **trois** valeurs, dites laquelle vous retenez et **pourquoi**. Une valeur retenue sans justification est une valeur arbitraire.
5. **Les limites.** Qu'est-ce que votre chaîne ne peut pas voir ? Distinguez ce qui vient de la relation comptée, ce qui vient du critère de liaison, et ce qui vient du seuil.

IL N'Y A PAS DE « BONNE » PARTITION ATTENDUE

Le barème ne récompense pas l'obtention d'une "architecture idéale", notion non-existante ici. Il récompense la **correction** du calcul, sa **reproductibilité**, la **justification** des choix et la **qualité de l'interprétation**.

PARTIE C Aller plus loin

BONUS OPTIONNEL · NON NÉCESSAIRE POUR TERMINER LE TP

LISEZ CECI AVANT DE COMMENCER

Cette partie est **entièrement facultative**. Les parties A et B permettent d'obtenir 20/20 sans elle, et aucune de ses pistes ne remplace une étape manquante des parties obligatoires. Ne l'abordez qu'une fois l'étape B4 franchie.

C1 Comparer le lien moyen et le lien simple

Ajoutez le **lien simple** (la similarité de deux groupes est celle de leur *meilleur* couple) en option, et comparez les deux arbres sur le même graphe de couplage : à partir de quelle étape divergent-ils, et pourquoi ?

C2 Enrichir la relation de dépendance

Ajoutez au moins une relation à celles qui sont comptées — héritage, implémentation d'interface, type d'un attribut, type d'un paramètre ou d'un retour, instanciation. **Comparez les modules obtenus** avec ceux du modèle imposé. Le bloc `extra` du jeu de données canonique fournit l'héritage ; les autres relations demandent d'étendre votre extracteur du TP 1 et de mettre à jour votre modèle de faits.

C3 Explorer les résultats dans une interface graphique

Ce bonus demande une **interface graphique interactive** par-dessus les artefacts que votre chaîne produit déjà — une **seconde interface, plus ergonomique, permettant d'explorer et de manipuler les résultats du même modèle**. Elle doit offrir au moins deux interactions utiles et justifiées : navigation dans l'arbre ou le graphe, recherche d'une classe, filtrage sur le couplage ou sur la taille des modules, sélection et inspection d'un nœud, d'un groupe ou d'un module, changement de seuil `CP` ou de vue. Aucune bibliothèque n'est imposée.

C4 Étendre l'analyse

Appliquez votre chaîne à un **second projet** Java de taille modérée, reconstruisez ses faits d'entrée avec votre extracteur du TP 1, avec **JDT** ou avec **Spoon**, puis comparez les modules obtenus à ceux issus des faits fournis. Discutez ce que l'écart doit au projet et ce qu'il doit à la méthode d'extraction.

6. Livrables et modalités de remise

Vous rendez **une seule archive par binôme**, nommée `NOM1_NOM2_HAI913I_TP2.zip` (en monôme : `NOM_Prenom_HAI913I_TP2.zip`), contenant les quatre livrables ci-dessous. Un livrable manquant, illisible ou inaccessible affecte directement les points correspondants du barème.

6.1. A — Compte-rendu (PDF)

Un **rapport d'ingénierie**, pas une documentation de code : il explique et justifie ce que vous avez fait. Ce n'est ni un listing des sources, ni de la Javadoc engendrée.

Page de titre : Université de Montpellier — Faculté des Sciences ; HAI913I — Évolution et restructuration des logiciels ; TP 2 — Compréhension et architectures ; enseignant responsable : Bachar Rima ; année universitaire 2026–2027 ; date ; puis, **pour chaque membre** : prénom, nom, numéro étudiant et parcours (GL ou ICO).

Contenu attendu, précédé d'une table des matières :

1. **Objectifs et périmètre** : ce que fait votre outil, ce qu'il ne fait pas.
2. **Conception générale** : l'architecture, des faits aux modules ; **le chemin d'entrée retenu** (§ 4) et l'interface qui l'isole.
3. **Conventions appliquées** : la relation comptée, la formule du couplage **et son dénominateur**, le critère de liaison, la règle de départage des égalités, la règle de coupe. Cette section doit permettre de **rejouer** vos résultats.
4. **Réponses étape par étape** (A0 à B4) : pour chacune, votre interprétation, la stratégie retenue, l'implémentation (extraits courts et ciblés), le résultat obtenu, la façon dont vous l'avez validé, les difficultés.
5. **Diagrammes** : classes du modèle, graphe de couplage, dendrogramme, enchaînement des traitements, les vues architecturales obtenues.
6. **Validation** : comparaison avec les points de contrôle A0 à B3, écarts et explications.
7. **Interprétation et regard critique** : les cinq points de l'étape B4, y compris les valeurs de `CP` testées et le choix retenu.
8. **Difficultés et parties incomplètes**, et comment vous avez essayé de les adresser.
9. **Utilisation des outils d'intelligence artificielle** (§ 3.1) : si vous en avez utilisé ; lesquels (outil, modèle) ; pour quelles tâches ; des exemples représentatifs de l'aide obtenue ; comment vous l'avez vérifiée ; en quoi les décisions restent les vôtres. *Si vous n'en avez pas utilisé, écrivez-le explicitement.* Ne joignez pas d'historique complet de conversation.
10. **Améliorations possibles** et **conclusion**.
11. **Références** : notes de bas de page pour un renvoi isolé, bibliographie pour les sources substantielles, dans un style de citation constant.

6.2. B — Code source

Le projet complet, qui se construit et s'exécute à partir des seules sources. **Sans** artefacts engendrés : pas de `target/`, `bin/` ni `out/`, pas de caches d'IDE ni de dépendances, pas de JAR, pas de fichiers temporaires ni de métadonnées système (`.DS_Store`, `__MACOSX`).

L'usage de **Git** est encouragé (un `.gitignore` adapté, des *commits* qui ont un sens, ni binaires ni identifiants). Si l'archive dépasse la limite affichée sur Moodle, fournissez un lien vers un **dépôt Git** (GitHub, GitLab ou équivalent stable). **Si vous utilisez un dépôt, son URL figure dans le compte-rendu** et, de préférence, dans le README et `liens.txt`. Évitez les partages de fichiers génériques ou temporaires ; en cas d'hébergement externe, l'équipe pédagogique doit y avoir accès sans démarche et le lien doit rester valide pendant toute la correction.

6.3. C — README (PDF)

Rendu **en PDF** pour faire partie du dossier évalué (un `README.md` peut rester dans le dépôt). Un lecteur qui ne connaît pas votre travail doit pouvoir **reconstruire, exécuter et vérifier** l'outil à partir de ce seul document.

RUBRIQUE	CONTENU
Projet	modules et structure, architecture, **chemin d'entrée retenu**, entrées attendues, sorties produites, dépendances éventuelles, limites connues
Environnement	version exacte du JDK, de Maven, des bibliothèques ajoutées le cas échéant, de Graphviz, système d'exploitation, **IDE et sa version, ou éditeur + outils en ligne de commande**
Construction	commandes exactes, depuis une copie fraîche
Exécution	commandes, arguments, **valeur de `CP`**, exemples complets
Validation	jeu de données utilisé, sorties attendues ou points de contrôle
Dépannage	erreurs fréquentes et leur remède

6.4. D — Vidéo de démonstration

Une démonstration de l'**outil finalisé, en cours d'exécution** — pas de diaporama. Elle montre, dans l'ordre qui vous convient :

- la construction du projet depuis un état propre ;
- le chargement des faits et les compteurs du point de contrôle A0 ;
- le couplage et le graphe pondéré ;
- le regroupement et le dendrogramme ;
- l'identification des modules, **pour au moins deux valeurs de `CP`**, dont une qui viole la contrainte $M/2$;
- la correspondance entre ce qui est montré et les étapes du sujet.

Commentaire oral facultatif ; des sous-titres ou du texte à l'écran conviennent. La vidéo n'a pas besoin d'être montée avec soin. Elle est incluse dans l'archive si sa taille le permet, placée dans le dépôt, ou déposée sur une plateforme stable (YouTube ou équivalent) ; son **URL figure alors dans le compte-rendu et dans le README**. Vous êtes responsables des droits d'accès.

DÉPÔT ET DATE LIMITE

Structure attendue de l'archive (adaptez-la si votre architecture le justifie) :

```
NOM1_NOM2_HAI913I_TP2.zip          (monome : NOM_Prenom_HAI913I_TP2.zip)
|-- Compte-Rendu.pdf
|-- README.pdf
|-- source/                         pom.xml, src/ ... (sans target/, caches)
|-- sorties/                        graphe, dendrogramme, modules (exports)
|-- video/
|   `-- demonstration.mp4           (ou liens.txt si la video est externe)
`-- liens.txt                       depot Git, video externe
```

DATE LIMITE — Consulter le dépôt Moodle **« TP2 — Compréhension et architectures »**. Toute modification du calendrier sera répercutée sur ce dépôt.

Il existe **un seul espace de dépôt** pour tous les groupes de HAI913I. Un seul dépôt par binôme, effectué par l'un des deux membres ; les deux noms figurent dans le nom de l'archive et sur la page de titre.

7. Barème

Le barème porte sur les **parties A et B** du sujet et totalise **20 points**. Chaque critère renvoie aux étapes du sujet. Les correcteurs exécutent aussi votre outil sur un **second jeu de faits de même nature** que celui du projet de validation, pour vérifier que les résultats ne dépendent pas de ce dernier.

CRITÈRE	ÉTAPES	CE QUI EST REGARDÉ	PTS
I. DES FAITS AU GRAPHE DE COUPLAGE — 5 points			
Chargement des faits et interface d'entrée	A0	un seul point d'entrée, les deux chemins possibles derrière	1
Agrégation et métrique de couplage	A1	arcs inter-classes, symétrie, dénominateur, somme égale à 1	2,5
Graphe pondéré et export relisible	A2	nœuds, arêtes, poids, classes isolées, rechargement	1,5
II. REGROUPEMENT, MODULES ET INTERPRÉTATION — 8 points			
Clustering ascendant et recalcul par lien moyen	B1	boucle de fusion, moyenne sur tous les couples, 14 fusions	3
Déterminisme et règle de départage	B1	deux exécutions identiques, égalités tranchées comme imposé	1
Dendrogramme : représentation et restitution	B2	arbre valide, ordre des fusions, similarités lisibles	1,5
Coupe sous les trois contraintes	B3	branche, couplage interne moyen $>$ `CP`, $M/2$ et son rejet	2,5
III. QUALITÉ D'INGÉNIERIE ET REPRODUCTIBILITÉ — 2 points			
Architecture et séparation des responsabilités	A0 – B3	entrée, couplage, regroupement, coupe, sortie : séparés et testables	1
Construction et exécution reproductibles, gestion des erreurs	A0 B3	copie fraîche, chemin invalide, `CP` mal formé, jeu de données absent	1
IV. COMMUNICATION — 5 points			
Compte-rendu : conventions, validation, justifications	—	structure, conventions rejouables, écarts expliqués	1,5
Interprétation des modules et regard critique	B4	les cinq points de B4, appuyés sur vos propres sorties	1,5
Vidéo de démonstration	—	outil finalisé en exécution, deux valeurs de `CP`	1,5
README, déclaration d'usage de l'IA et références	—	un tiers reconstruit et vérifie ; déclaration présente et honnête	0,5
Total			20

PARTIE C — TRAITEMENT DES BONUS

Les bonus viennent **compenser** des points perdus sur les parties A et B. **Leur total est plafonné à +4 points, et la note finale à 20/20.**

BONUS	PTS
C1 — lien simple implémenté *et* divergence des deux arbres expliquée	**+1**
C2 — relation de dépendance enrichie *et* modules comparés	**+1**
C3 — interface graphique interactive par-dessus le graphe ou le dendrogramme	**+1**
C4 — second projet analysé, faits reconstruits, modules comparés	**+1**
Total pris en compte	**+4 max.**

Les bonus ne sont attribués que si la chaîne fonctionne sur le projet de validation (points de contrôle A1, B1 et B3 franchis pour l'essentiel).

CE QUI N'EST PAS ÉVALUÉ

La mise en forme décorative du compte-rendu, le nombre de diagrammes, la longueur du rendu, le nombre de *commits*, le temps d'exécution, et le fait d'avoir ou non utilisé un assistant d'IA. À l'inverse, une incohérence entre le compte-rendu, le code et la vidéo affecte les critères correspondants.

8. Récapitulatif

À VÉRIFIER AVANT DE DÉPOSER

- ☐ Les points de contrôle A0 à B3 sont franchis, ou les écarts expliqués dans le compte-rendu.
- ☐ Les conventions de § 5 sont appliquées et **écrites** dans le compte-rendu ; toute variante supplémentaire est nommée et définie.
- ☐ Deux exécutions successives produisent exactement la même sortie.
- ☐ Au moins trois valeurs de `CP` ont été essayées, dont une qui viole la contrainte $M/2$, et le choix final est justifié.
- ☐ L'étape B4 est traitée : modules contre paquetages, classes isolées, un groupe discuté, effet de `CP`, limites de la relation comptée.
- ☐ L'outil se construit depuis une copie fraîche, sans IDE, avec les commandes du README, et refuse proprement les entrées invalides.
- ☐ `Compte-Rendu.pdf` — page de titre des deux membres, table des matières, conventions, étapes, validation, interprétation, section IA, références.
- ☐ `README.pdf` — projet, environnement, construction, exécution, validation, dépannage.
- ☐ Vidéo montrant l'outil en exécution, accessible ; URL dans le compte-rendu si externe.
- ☐ URL du dépôt Git dans le compte-rendu, le cas échéant.
- ☐ Archive `NOM1_NOM2_HAI913I_TP2.zip`, sans `target/`, `bin/`, caches ni JAR.
- ☐ Dépôt dans l'espace Moodle « **TP2 — Compréhension et architectures** », avant l'échéance qui y est affichée.